Approximation algorithms solutions

35-E01 Matching lower bounds and a weighted counterexample

Selecting the center c and any leaf a covers all edges. Unweighted ALG=2 and OPT=1, attaining ratio 2, with matching lower bound 1. The same output has weight 101, while center alone costs 1, for weighted ratio 101.

A matching edge forces a cover to choose at least one endpoint, but does not ensure the algorithm's two endpoint weights are at most twice the weight of the optimal chosen endpoint. Replacing leaf weight by arbitrary W gives ratio W+1 with no constant bound. Counting two endpoints versus at least one controls cardinality, not weighted sums. Weighted approximation requires a weight-sensitive lower bound and rule, such as LP rounding.

35-E02 Metric walks and tours

The MST chain has AD=2, DC=3, CB=2, totaling 7. All six costs are 2,2,3,3,5,5. Every spanning tree uses three edges and costs at least 2+2+3=7, attained by this chain.

The full walk A,D,C,B,C,D,A costs 2+3+2+2+3+2=14. The first-visit tour A,D,C,B,A costs 2+3+2+3=10. A closed tour spanning x=0 to x=3 has total x variation at least 6. Spanning y=0 to y=2 needs total y variation at least 4. Every Manhattan tour costs at least 10, attained here, so OPT=10.

Walk 14, tour 10, and OPT 10 are distinct. A claim OPT=6 violates both MST$\leq$OPT, whose lower bound is 7, and the coordinate-variation lower bound 10. Inconsistent data cannot be accepted before applying a factor-two inequality.

35-E03 Weighted set-cover charges

Initially the cost per new element is 2/3,2/3,1/4 for S1,S2,S3. Choose S3. Elements 1,2,4,5 each pay 1/4, leaving {3,6}. Next, S1,S2 each cover one new element at cost per element 2. Choose S1, charge element 3 a cost of 2, and leave {6}. Finally choose S2 and charge element 6 a cost of 2. ALG=5.

S1,S2 together cost 4 and cover X. Any cover using S3 still needs S1 for 3 and S2 for 6, costing 5. No single set is feasible, so OPT=4. Charges on S1 total 1/4+1/4+2=5/2, and S2 has the same total. Each is at most $w(S)H_3 = 2 \cdot 11/6 = 11/3$.

The actual ratio is 5/4, at most $H_4 = 25/12$, since the largest family set has size 4. Charges sum to actual cost 5. Summing over optimum sets gives $5 \leq H_3 \cdot 4 \leq H_4 \cdot 4$, with no omitted element.

Approximation algorithms solutions (continued)

35-E04 Expected objective versus expected ratio

If c=1, all four clauses are true and Y=4. If c=0, exactly one of the four sign combinations of a,b is false, so Y=3. Each case has probability 1/2. Thus E[Y]=7/2 and OPT=4, but

$$E[OPT/Y] = \tfrac{1}{2}(4/4) + \tfrac{1}{2}(4/3) = 7/6 > 8/7.$$

This disproves interpreting the expected-objective 8/7 guarantee as an expected-ratio bound.

For conditional expectation, a=0 and a=1 both give 7/2, so choose 0. Given a=0, b=0 and b=1 also both give 7/2, so choose 0. Now c=0 gives 3 and c=1 gives 4. Choose c=1, yielding assignment (0,0,1) satisfying all four clauses.

The clause events share c and are not independent. Linearity of expectation never required that independence. Independence of the three distinct variables within a clause is what gives all-false probability 1/8.

35-E05 LP bound, integer optimum, and rounding

Multiply edge constraints ab,ac,bc by 1,2,3 respectively. Their sum has left side $3x_a + 4x_b + 5x_c$ and right side 6. Nonnegative multipliers preserve the inequalities, so every feasible LP solution costs at least 6. The half-value solution has cost (3+4+5)/2=6 and is optimal.

An integer cover needs two vertices, with possible costs 7,8,9, so OPT=7. Threshold at least 1/2 takes all three, costing ALG=12=2LP*≤2OPT=14. The actual ratio is 12/7. The exact weighted inequality proves the LP lower bound; visual symmetry alone would not prove it.

35-E06 Four-round FPTAS integer trace

Here n=4 and δ=(2/5)/8=1/20.

i	Merged unique	After TRIM	After removing values above 305
1	0,102	0,102	0,102
2	0,98,102,200	0,98,200	0,98,200
3	0,98,195,200,293,395	0,98,195,293,395	0,98,195,293
4	0,98,101,195,199,293,296,394	0,98,195,293,394	0,98,195,293

Approximation algorithms solutions (continued)

For example, 102≤98・1.05=102.9, 200≤195・1.05=204.75, and 296≤293・1.05=307.65, so each is omitted in favor of a smaller representative. The returned subset uses indices 2,3 and value 293=98+195.

The exact feasible sums are {0,98,101,102,195,199,200,203,293,296,297,301}. Thus OPT=301=102+98+101. Feasibility is 293≤305, and 301/293≈1.0273≤1.4. Also 293≥0.6・301. Every retained representative is an actual subset sum. Epsilon is global; only δ=1/20 controls each round.

35-E07 Representative direction and negative items

After 4,5, the exact sums are 0,4,5,9. Removing only 4 in favor of larger 5 leaves 0,5,9. Adding 6 yields candidates 0,5,6,9,11,15. After capacity filtering, the largest is 9. The actual optimum 4+6=10 has been lost. The induction needs z≤y so later addition remains within capacity whenever y would. A larger representative lacks this property.

For 8,-3 with t=5, removing 8 after round 1 leaves only 0. The next step obtains 0 or -3 but loses 8+(-3)=5. An oversized sum cannot recover with positive future terms, but can recover with negative terms.

These examples refute direct extension of the existing operation and proof. They do not prove every negative-item or alternative-representation algorithm impossible to approximate. A changed model requires new feasibility, state, and error analysis.

35-E08 Gap reductions and adding constants

For n=5 and ρ=2, use nonedge weight M=2n+1=11. With an HC, OPT=5, so the assumed approximation returns cost at most 10. Without an HC, every tour uses an edge of cost 11 plus four edges of cost at least 1, totaling at least 15>10. Threshold 10 separates the cases. The value 11 has four binary bits and creates no excessive output length.

For arbitrary nonnegative costs c, let K=max c and $c'(u,v) = c(u,v) + K$. Since $c(u,w) \le K$ and $c(u,v) + c(v,w) \ge 0$,

$$c(u,w) + K \le 2K \le c(u,v) + c(v,w) + 2K.$$

Thus c′ is metric. Every n-vertex tour gains exactly nK, so the set of optimal tours is unchanged.

However, $c'(H) \le \rho OPT'$ only yields

$$c(H) + nK \le \rho(OPT + nK), \qquad c(H) \le \rho OPT + (\rho - 1)nK.$$

Approximation algorithms solutions (continued)

The extra term can dwarf OPT, so the original ratio does not follow. Preserving optimal solutions need not preserve approximation ratios. Adding a constant to obtain a metric is consistent with the general-TSP gap hardness argument.

Verification code appendix: approximation_lab.py

```
"""Exact arithmetic models for approximation ratios, charges and TRIM witnesses."""
from fractions import Fraction
from dataclasses import dataclass

def maximal_cover(vertices,edges):
    chosen=set();matching=[]
    for u,v in edges:
        if u not in chosen and v not in chosen:
            chosen.update((u,v));matching.append((u,v))
    return chosen,matching

def greedy_cover(universe,sets,weights):
    remaining=set(universe);picked=[];charges={};trace=[]
    while remaining:
        candidates=[(Fraction(weights[i],len(s&remaining)),i) for i,s in enumerate(sets) if
s&remaining]
        if not candidates:raise ValueError('family does not cover the universe')
        price,i=min(candidates);new=sets[i]&remaining
        for e in new:charges[e]=price
        remaining-=new;picked.append(i);trace.append((i,sorted(new),sorted(remaining),price))
    return picked,charges,trace

@dataclass(frozen=True)
class Witness:
    value:int
    parent:object=None
    index:int=-1

def merge_nodes(a,b):
    i=j=0;out=[]
    while i<len(a) or j<len(b):
        if j==len(b) or (i<len(a) and a[i].value<=b[j].value):v=a[i];i+=1
        else:v=b[j];j+=1
        if not out or out[-1].value!=v.value:out.append(v)
    return out

def subset_trim(values,target,epsilon,collect_trace=False):
    if not 0<epsilon<1:raise ValueError('epsilon in(0,1) required')
    if target<0 or any(v<=0 for v in values):raise ValueError('nonnegative target and
positive values required')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Approximation algorithms solutions (continued)

```
    if not values or target==0:return 0,[],[]
    delta=Fraction(epsilon)/(2*len(values));current=[Witness(0)];trace=[]
    for i,value in enumerate(values):
        added=[Witness(p.value+value,p,i) for p in current] if value<=target else []
        merged=merge_nodes(current,added);trimmed=[]
        for p in merged:
            if not trimmed or p.value>(1+delta)*trimmed[-1].value:trimmed.append(p)
        current=[p for p in trimmed if p.value<=target]
        if collect_trace:trace.append((i+1,[p.value for p in merged],[p.value for p in
trimmed],[p.value for p in current]))
    best=current[-1];value=best.value;indices=[]
    while best.parent is not None:indices.append(best.index);best=best.parent
    return value,indices[::-1],trace

def exact_subset(values,target):
    sums={0}
    for x in values:sums|={y+x for y in sums if y+x<=target}
    return sorted(sums)

def conditional_max3(clauses,variables):
    def expectation(assignment):
        total=Fraction(0)
        for clause in clauses:
            if any(abs(l) in assignment and assignment[abs(l)]==(l>0) for l in
clause):total+=1
            else:
                r=sum(abs(l) not in assignment for l in clause)
                total+=1-Fraction(1,2**r)
        return total
    assignment={};trace=[]
    for v in variables:
        left=expectation(assignment|{v:False});right=expectation(assignment|{v:True})
        assignment[v]=right>left;trace.append((v,left,right,assignment[v]))
    return assignment,expectation(assignment),trace

if __name__=='__main__':
    print('set cover',greedy_cover(set(range(1,7)),[{1,2,3},{4,5,6},{1,2,4,5}],[1,1,1]))
    print('subset',subset_trim([112,108,205,103],325,Fraction(2,5),True))
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study